PROJECT REPORT ON

WebScan-Lite: A Lightweight Python Tool for Automated Website Vulnerability Assessment

Submitted in partial fulfilment of the requirements for the degree of
CEH (Certified Ethical Hacker)

Submitted By:
Prem Parshuram Shelavale

Boston University
(BIA - Boston Institute of Analytics)
Batch: June 2026

DECLARATION

I hereby declare that the project report entitled **"WebScan-Lite"** submitted by me is the result of my own work. The findings and conclusions are based on the research and practical implementation conducted by me. I have taken due care to acknowledge all sources of information used in this project.

Prem Parshuram Shelavale

Date:

ACKNOWLEDGEMENT

I would like to express my deepest appreciation to my guide, [Guide Name], for their continuous support and guidance. I also thank Boston University for providing the necessary resources. Finally, I would like to thank OWASP for creating the Juice Shop application, which made my Proof of Concept possible in a completely legal and ethical environment.

ABSTRACT

Web application security misconfigurations remain one of the most prevalent and easily exploitable categories of vulnerabilities, consistently ranking high in the OWASP Top 10. Manual audits of HTTP security headers, transport-layer configuration, and information disclosure are time-consuming and prone to human error.

This project presents **WebScan-Lite**, a lightweight, non-intrusive, single-file Python tool that automates the detection of eight categories of common web security weaknesses: HTTPS enforcement, TLS certificate health, HTTP security headers, technology/version disclosure, cookie hardening, dangerous HTTP methods, sensitive file exposure, and directory listing.

The tool performs a passive assessment of a target site, generates a severity-ranked findings report mapped to OWASP Top 10:2021 categories, computes a 0–100 security score, and exports structured JSON suitable for CI/CD integration. The Proof of Concept, conducted against the OWASP Juice Shop training application within a Kali Linux environment, successfully identified missing security headers and technology disclosures. The final output is an actionable remediation report that enables development teams to baseline and improve their web security posture.

TABLE OF CONTENTS

Sr. No.	Topic	Page No.
1	Abstract	5
2	Chapter 1: Introduction & Objectives	7
3	Chapter 2: Research & Data Collection	8
4	Chapter 3: Impact Analysis (BIA)	9
5	Chapter 4: Tool Design & Methodology	10
6	Chapter 5: Proof of Concept (Results)	11
7	Chapter 6: Recommendations & Conclusion	12
8	References	13
9	Appendix (Source Code)	14

CHAPTER 1: INTRODUCTION & OBJECTIVES

1.1 Introduction

The purpose of this project is to design a non-intrusive, easy-to-use Python scanner for common web misconfigurations. Web applications face numerous threats, and many organizations fail to implement basic security controls like HTTP security headers. This tool bridges that gap.

1.2 Objectives

1. Design a non-intrusive, easy-to-use Python scanner for common web misconfigurations.
2. Map every finding to an OWASP Top 10:2021 category for contextual remediation.
3. Produce a quantified security score to enable pre-release gating.
4. Provide a reproducible, ethical Proof of Concept against a deliberately vulnerable target.
5. Export machine-readable JSON for CI/CD integration.

1.3 Scope

- In-scope: Passive HTTP header observation, TLS certificate inspection, sensitive path exposure, HTTP method advertisement.
- Out of scope: Injection testing (SQLi, XSS payloads), authentication bypass, business-logic flaws, active fuzzing.

CHAPTER 2: RESEARCH & DATA COLLECTION

2.1 Target Site Profile

To ensure the Proof of Concept is 100% legal and ethical, the target chosen is a deliberately vulnerable application.

Field	**Detail**
Name	OWASP Juice Shop
URL	http://localhost:3000 (Self-hosted via Docker)
Category	Deliberately vulnerable web application
Why chosen	Legal to scan, reproducible, no legal risk

Ethical Note: Scanning third-party sites without written authorisation violates India's IT Act 2000. A local application was chosen to ensure full legal compliance.

2.2 Data Collection (Technology Stack)

Data collection was performed fully passively by the tool. No port scanning or fingerprinting payloads were sent.

Layer	**Technology**	**Detection Method**
Web server	Node.js + Express	Server, X-Powered-By header
Frontend	Angular SPA	JS bundle references
Database	SQLite	Documented stack
Auth	JWT-based	Authorization header flow
Headers	CSP absent	Tool output
Cookies	Secure flag absent	Tool output

CHAPTER 3: IMPACT ANALYSIS (BIA)

BIA Element	Assessment
Business process at risk	Public web portal / customer-facing application
Vulnerabilities found	Missing CSP, missing HSTS, version disclosure, weak cookies
Business impact	Enlarged attack surface — XSS, clickjacking, session hijack
Financial impact (est.)	₹2–5 lakh/day of downtime (illustrative)
Compliance impact	OWASP ASVS, PCI-DSS 6.5, ISO 27001 A.14, India DPDP Act 2023
RTO / RPO	RTO: 4 hours (config fixes are low effort) / RPO: 24 hours

CHAPTER 4: TOOL DESIGN & METHODOLOGY

4.1 Architecture

[CLI / Config] → [HTTP Session] → [8 Check Modules] → [Scoring Engine] → [Console + JSON Report]

4.2 Detection Modules

1. **HTTPS Enforcement:** Checks for HTTP→HTTPS redirects.
2. **TLS Health:** Checks for expired certificates or TLS 1.0/1.1.
3. **Security Headers:** Checks for CSP, HSTS, XFO, XCTO, RP, PP.
4. **Version Disclosure:** Checks for Server / X-Powered-By version leaks.
5. **Cookie Hardening:** Checks for missing Secure / HttpOnly / SameSite.
6. **HTTP Methods:** Checks if TRACE, PUT, or DELETE are advertised.
7. **Sensitive Files:** Checks for .env, .git/HEAD, backups using Soft-404 baseline.
8. **Directory Listing:** Checks if auto-index is enabled.

4.3 Scoring Logic

Starts at 100; deducts Critical (−25), High (−15), Medium (−8), Low (−3). Grades: A ≥ 85, B ≥ 70, C ≥ 50, D otherwise.

CHAPTER 5: PROOF OF CONCEPT & RESULTS

The tool was executed against OWASP Juice Shop (http://localhost:3000) within a Kali Linux environment using the command:

```
python3 webscan_lite.py http://localhost:3000 --authorized --json report.json
```

Observed findings:

- [HIGH] HDR-content-security-policy — Missing CSP header.
- [MEDIUM] HDR-strict-transport-security — Missing HSTS.
- **Computed score:** 58/100 (Grade C).

Figure 1: Overall Security Score and Grade

```
             Evidence: Normal document

================================================================================
  SECURITY SCORE: 56/100  (Grade C)
================================================================================
  HIGH: 2 | MEDIUM: 1 | LOW: 2

  JSON report written to: report.json
```

Figure 2: Missing Security Headers detected by the tool

```
--------------------------------------------------------------------------------

  [HIGH    ] HTTPS-01 - Site served over plain HTTP
               OWASP    : A02 Cryptographic Failures
               Evidence: Scheme='http'
               Fix      : Serve all traffic over HTTPS and 301-redirect HTTP→HTTPS.

  [HIGH    ] HDR-content-security-policy - Missing header: content-security-policy
               OWASP    : A05 Security Misconfiguration
               Evidence: Header absent
               Fix      : Add: Content-Security-Policy: default-src 'self'
```

Figure 3: High Severity Vulnerability Evidence and Remediation

```
               Evidence: Header absent
               Fix      : Add: Content-Security-Policy: default-src 'self'

  [MEDIUM  ] HDR-strict-transport-security - Missing header: strict-transport-security
               OWASP    : A02 Cryptographic Failures
               Evidence: Header absent
               Fix      : Add: Strict-Transport-Security: max-age=31536000; includeSubDomains

  [LOW     ] HDR-referrer-policy - Missing header: referrer-policy
               OWASP    : A05 Security Misconfiguration
```

CHAPTER 6: RECOMMENDATIONS & CONCLUSION

6.1 Recommendations

- **Immediate:** Nginx: add_header Content-Security-Policy "default-src 'self'" always; server_tokens off; TraceEnable off.
- **Process:** Integrate WebScan-Lite into CI/CD pipelines; fail builds if score < 80. Maintain approved headers as Infrastructure-as-Code.
- **Defence in Depth:** Deploy WAF/CDN (Cloudflare, AWS WAF). Enforce HTTPS-only with HSTS preload. Automate certificate renewal.

6.2 Conclusion

WebScan-Lite demonstrates that a single-file Python tool with one dependency can meaningfully contribute to a web application's security baseline. By automating repetitive checks — headers, TLS, disclosure, and exposure — it frees human effort for the checks that genuinely require judgement: access-control logic and business-logic flaws. The tool is intentionally non-intrusive, legal to run in authorized environments, and CI/CD-friendly.

REFERENCES

1. OWASP Top 10 — https://owasp.org/Top10/
2. OWASP HTTP Security Response Headers Cheat Sheet
3. OWASP Juice Shop — https://owasp.org/www-project-juice-shop/
4. MDN Web Docs — HTTP Headers
5. Python requests documentation — https://requests.readthedocs.io/
6. NIST SP 800-115 — Technical Guide to Information Security Testing
7. CWE-16: Configuration

APPENDIX: SOURCE CODE

File Name: webscan_lite.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
WebScan-Lite v1.0 — Non-intrusive web vulnerability scanner
Author: Prem Parshuram Shelavale | CEH June 2026 | Boston University

Usage:
  python webscan_lite.py http://localhost:3000 --authorized --json report.json
"""
from __future__ import annotations
import argparse, json, re, socket, ssl, sys, time, uuid
from datetime import datetime, timezone
from urllib.parse import urljoin, urlparse

try:
    import requests
    from requests.exceptions import RequestException
except ImportError:
    sys.exit("Missing dependency. Run: pip install requests")

DEFAULT_TIMEOUT, DEFAULT_DELAY, MAX_BODY_BYTES = 10, 0.3, 20000
USER_AGENT = "WebScan-Lite/1.0 (authorized security assessment)"
SEVERITY_WEIGHT = {"CRITICAL": 25, "HIGH": 15, "MEDIUM": 8, "LOW": 3, "INFO": 0}

SECURITY_HEADERS = {
    "content-security-policy":   ("HIGH",   "A05 Security Misconfiguration",
                                  "Add: Content-Security-Policy: default-src 'self'"),
    "strict-transport-security": ("MEDIUM", "A02 Cryptographic Failures",
                                  "Add: Strict-Transport-Security: max-age=31536000; includeSubDomains"),
    "x-content-type-options":    ("MEDIUM", "A05 Security Misconfiguration",
                                  "Add: X-Content-Type-Options: nosniff"),
    "x-frame-options":           ("MEDIUM", "A05 Security Misconfiguration",
                                  "Add: X-Frame-Options: SAMEORIGIN"),
    "referrer-policy":           ("LOW",    "A05 Security Misconfiguration",
                                  "Add: Referrer-Policy: strict-origin-when-cross-origin"),
    "permissions-policy":        ("LOW",    "A05 Security Misconfiguration",
```

```
                                    "Add: Permissions-Policy: geolocation=(), camera=()"),
}
LEAKY_HEADERS = ["server", "x-powered-by", "x-aspnet-version", "x-generator"]

SENSITIVE_PATHS = [
    ("/.env",           "CRITICAL", "Environment file with secrets"),
    ("/.git/HEAD",      "CRITICAL", "Exposed Git repository metadata"),
    ("/config.php.bak", "HIGH",     "Backup of configuration file"),
    ("/backup.zip",     "HIGH",     "Archive backup"),
    ("/db.sql",         "CRITICAL", "Database dump"),
    ("/phpinfo.php",    "MEDIUM",   "PHP configuration disclosure"),
    ("/.DS_Store",      "LOW",      "macOS metadata file"),
    ("/server-status",  "MEDIUM",   "Apache server-status page"),
]

class C:
    ON = sys.stdout.isatty()
    RED = "\033[91m" if ON else ""; GRN = "\033[92m" if ON else ""
    YEL = "\033[93m" if ON else ""; BLU = "\033[94m" if ON else ""
    CYN = "\033[96m" if ON else ""; BLD = "\033[1m"  if ON else ""
    RST = "\033[0m"  if ON else ""

SEV_COLOR = {"CRITICAL": C.RED, "HIGH": C.RED, "MEDIUM": C.YEL, "LOW": C.CYN, "INFO": C.BLU}

def normalize(u):
    u = u.strip()
    return u if u.startswith(("http://", "https://")) else "https://" + u

def build_session(t, insecure):
    s = requests.Session()
    s.headers.update({"User-Agent": USER_AGENT, "Accept": "*/*"})
    s.verify = not insecure
    s.max_redirects = 5
    s.request_timeout = t
    return s

def add(f, fid, title, sev, cat, ev, fix):
    f.append({"id": fid, "title": title, "severity": sev,
              "owasp": cat, "evidence": ev, "remediation": fix})
```

```
def check_https(session, url, f):
    p = urlparse(url)
    if p.scheme != "https":
        add(f, "HTTPS-01", "Site served over plain HTTP", "HIGH",
            "A02 Cryptographic Failures", f"Scheme='{p.scheme}'",
            "Serve all traffic over HTTPS and 301-redirect HTTP→HTTPS.")
        return
    http_url = f"http://{p.netloc}{p.path or '/'}"
    try:
        r = session.get(http_url, timeout=session.request_timeout, allow_redirects=False)
    except RequestException as e:
        add(f, "HTTPS-02", "HTTP endpoint unreachable", "INFO",
            "A02 Cryptographic Failures", str(e), "No action required.")
        return
    loc = r.headers.get("Location", "")
    if r.status_code in (301, 302, 307, 308) and loc.lower().startswith("https://"):
        add(f, "HTTPS-OK", "HTTP correctly redirects to HTTPS", "INFO",
            "A02 Cryptographic Failures", f"{r.status_code} → {loc}", "No action required.")
    else:
        add(f, "HTTPS-03", "HTTP does not redirect to HTTPS", "MEDIUM",
            "A02 Cryptographic Failures",
            f"GET {http_url} → {r.status_code}, Loc='{loc or 'none'}'",
            "Return 301 to HTTPS for every HTTP request.")


def check_tls(host, f, t):
    ctx = ssl.create_default_context()
    try:
        with socket.create_connection((host, 443), timeout=t) as s:
            with ctx.wrap_socket(s, server_hostname=host) as ss:
                cert = ss.getpeercert(); proto = ss.version()
    except ssl.SSLCertVerificationError as e:
        add(f, "TLS-01", "TLS certificate verification failed", "CRITICAL",
            "A02 Cryptographic Failures", e.verify_message,
            "Install a valid CA-signed certificate.")
        return
    except Exception as e:
        add(f, "TLS-02", "TLS handshake failed", "INFO",
            "A02 Cryptographic Failures", str(e), "Verify port 443 is reachable.")
```

```
        return
    try:
        exp = ssl.cert_time_to_seconds(cert["notAfter"])
        days = (exp - time.time()) / 86400
        es = datetime.fromtimestamp(exp, timezone.utc).strftime("%Y-%m-%d")
        if days < 0:
            add(f, "TLS-03", "Certificate EXPIRED", "CRITICAL",
                "A02 Cryptographic Failures", f"Expired {es}",
                "Renew immediately; automate renewal.")
        elif days < 30:
            add(f, "TLS-04", "Certificate expiring soon", "HIGH",
                "A02 Cryptographic Failures", f"Expires {es} ({days:.0f}d)",
                "Renew and automate renewal.")
        else:
            add(f, "TLS-OK", f"Certificate valid until {es}", "INFO",
                "A02 Cryptographic Failures", f"{days:.0f} days remaining",
                "No action required.")
    except Exception:
        pass
    if proto in ("TLSv1", "TLSv1.1"):
        add(f, "TLS-05", f"Deprecated protocol {proto}", "HIGH",
            "A02 Cryptographic Failures", f"Negotiated {proto}",
            "Disable TLS 1.0/1.1; allow only TLS 1.2+.")
    else:
        add(f, "TLS-OK2", f"Modern protocol ({proto})", "INFO",
            "A02 Cryptographic Failures", proto, "No action required.")

def check_headers(resp, f):
    hdrs = {k.lower(): v for k, v in resp.headers.items()}
    for name, (sev, cat, fix) in SECURITY_HEADERS.items():
        if name in hdrs:
            add(f, f"HDR-OK-{name}", f"'{name}' present", "INFO", cat,
                hdrs[name][:120], "No action required.")
        else:
            add(f, f"HDR-{name}", f"Missing header: {name}", sev, cat,
                "Header absent", fix)

def check_disclosure(resp, f):
    hdrs = {k.lower(): v for k, v in resp.headers.items()}
```

```
    for h in LEAKY_HEADERS:
        if h in hdrs:
            v = hdrs[h]
            if re.search(r"\d+\.\d+", v):
                add(f, f"DISC-{h}", f"Version disclosed via '{h}'", "MEDIUM",
                    "A05 Security Misconfiguration", f"{h}: {v}",
                    "Suppress version (Nginx: server_tokens off; Apache: ServerTokens Prod).")
            else:
                add(f, f"DISC-{h}", f"Technology disclosed via '{h}'", "LOW",
                    "A05 Security Misconfiguration", f"{h}: {v}",
                    "Consider removing the header.")

def get_set_cookies(resp):
    try:
        return resp.raw.headers.getlist("Set-Cookie")
    except Exception:
        sc = resp.headers.get("Set-Cookie")
        return [sc] if sc else []

def check_cookies(resp, f):
    cs = get_set_cookies(resp)
    if not cs:
        add(f, "COOK-00", "No cookies on landing page", "INFO",
            "A07 Auth Failures", "No Set-Cookie", "No action required.")
        return
    for raw in cs:
        name = raw.split("=", 1)[0].strip(); low = raw.lower()
        miss = [x for x in ("Secure", "HttpOnly", "SameSite") if x.lower() not in low]
        if miss:
            add(f, f"COOK-{name}", f"Cookie '{name}' missing: {', '.join(miss)}",
                "HIGH" if "Secure" in miss else "MEDIUM",
                "A07 Auth Failures", raw[:150],
                "Set Secure; HttpOnly; SameSite=Strict on session cookies.")
        else:
            add(f, f"COOK-OK-{name}", f"Cookie '{name}' hardened", "INFO",
                "A07 Auth Failures", raw[:120], "No action required.")

def check_methods(session, url, f):
    try:
```

```
        r = session.request("OPTIONS", url, timeout=session.request_timeout, allow_redirects=False)
        allow = r.headers.get("Allow", "")
        risky = [m for m in ("PUT", "DELETE", "TRACE", "CONNECT", "PATCH") if m in allow.upper()]
        if risky:
            add(f, "MTH-01", f"Risky methods: {', '.join(risky)}", "MEDIUM",
                "A05 Security Misconfiguration", f"Allow: {allow}",
                "Restrict to GET, POST, HEAD (OPTIONS).")
        else:
            add(f, "MTH-OK", "No risky HTTP methods", "INFO",
                "A05 Security Misconfiguration", f"Allow: {allow or 'not set'}",
                "No action required.")
    except RequestException as e:
        add(f, "MTH-00", "OPTIONS probe failed", "INFO",
            "A05 Security Misconfiguration", str(e), "No action required.")
    try:
        r = session.request("TRACE", url, timeout=session.request_timeout, allow_redirects=False)
        if r.status_code == 200 and "TRACE" in r.text.upper()[:2000]:
            add(f, "MTH-02", "TRACE method enabled (XST risk)", "HIGH",
                "A05 Security Misconfiguration", f"HTTP {r.status_code}",
                "Disable TRACE (Apache: TraceEnable off).")
    except RequestException:
        pass


def check_sensitive_files(session, base, f, delay):
    probe = f"/wsl-{uuid.uuid4().hex[:10]}.txt"
    try:
        b = session.get(urljoin(base, probe), timeout=session.request_timeout, allow_redirects=False)
        bs, bl = b.status_code, len(b.content)
    except RequestException:
        bs, bl = 404, 0
    hits = 0
    for path, sev, desc in SENSITIVE_PATHS:
        time.sleep(delay)
        try:
            r = session.get(urljoin(base, path), timeout=session.request_timeout, allow_redirects=False)
        except RequestException:
            continue
        if r.status_code == bs and abs(len(r.content) - bl) < 64:
            continue
```

```
        if r.status_code == 200:
            hits += 1
            snip = r.text[:80].replace("\n", " ")
            add(f, f"EXP-{path}", f"Sensitive file exposed: {path} ({desc})",
                sev, "A01 Broken Access Control",
                f"HTTP 200, {len(r.content)}B → {snip!r}",
                f"Block access to {path} at web server/WAF; remove from document root.")
    if hits == 0:
        add(f, "EXP-OK", "No sensitive files exposed", "INFO",
            "A01 Broken Access Control", f"Checked {len(SENSITIVE_PATHS)} paths",
            "No action required.")

def check_directory_listing(resp, f):
    body = resp.text[:MAX_BODY_BYTES].lower()
    ctype = resp.headers.get("Content-Type", "").lower()
    if resp.status_code == 200 and "text/html" in ctype:
        if "<title>index of" in body or "directory listing for" in body:
            add(f, "DIR-01", "Directory listing enabled", "MEDIUM",
                "A05 Security Misconfiguration",
                "Title shows 'Index of' / 'Directory listing for'",
                "Disable auto-index (Nginx: autoindex off; Apache: Options -Indexes).")
        else:
            add(f, "DIR-OK", "No directory listing on root", "INFO",
                "A05 Security Misconfiguration", "Normal document",
                "No action required.")

def score(fs):
    return max(0, 100 - sum(SEVERITY_WEIGHT[x["severity"]] for x in fs))

def grade(s):
    if s >= 85: return "A", C.GRN
    if s >= 70: return "B", C.GRN
    if s >= 50: return "C", C.YEL
    return "D", C.RED

def print_report(target, fs, el):
    total = score(fs); g, gc = grade(total)
    print("\n" + "=" * 74)
    print(f"{C.BLD}{C.CYN}  WebScan-Lite v1.0 — Vulnerability Assessment Report{C.RST}")
```

```
    print("=" * 74)
    print(f"  Target     : {target}")
    print(f"  Scanned at : {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
    print(f"  Duration   : {el:.1f}s")
    print(f"  Findings   : {len(fs)}")
    print("-" * 74)
    order = {"CRITICAL":0,"HIGH":1,"MEDIUM":2,"LOW":3,"INFO":4}
    for x in sorted(fs, key=lambda z: order[z["severity"]]):
        col = SEV_COLOR[x["severity"]]
        print(f"\n  {col}[{x['severity']:<8}]{C.RST} {C.BLD}{x['id']}{C.RST} — {x['title']}")
        print(f"              OWASP   : {x['owasp']}")
        print(f"              Evidence: {x['evidence']}")
        if x["severity"] != "INFO":
            print(f"              Fix     : {x['remediation']}")
    print("\n" + "=" * 74)
    print(f"  {C.BLD}SECURITY SCORE: {gc}{total}/100  (Grade {g}){C.RST}")
    print("=" * 74)
    cnt = {}
    for x in fs:
        if x["severity"] != "INFO":
            cnt[x["severity"]] = cnt.get(x["severity"], 0) + 1
    print("  " + (" | ".join(f"{k}: {v}" for k, v in cnt.items()) if cnt else "No actionable findings."))
    print()


def main():
    ap = argparse.ArgumentParser(description="WebScan-Lite — non-intrusive scanner")
    ap.add_argument("target")
    ap.add_argument("--authorized", action="store_true")
    ap.add_argument("--timeout", type=int, default=DEFAULT_TIMEOUT)
    ap.add_argument("--delay", type=float, default=DEFAULT_DELAY)
    ap.add_argument("--insecure", action="store_true")
    ap.add_argument("--json", metavar="FILE")
    a = ap.parse_args()
    print(f"{C.YEL}[!] WebScan-Lite must only be used on systems you own or have")
    print(f"    written permission to test.{C.RST}")
    if not a.authorized:
        if input("    Type 'yes' to confirm authorisation: ").strip().lower() != "yes":
            sys.exit("Aborted — authorisation not confirmed.")
    target = normalize(a.target); host = urlparse(target).hostname
```

```
    print(f"\n{C.CYN}[*]{C.RST} Scanning {target}\n")
    session = build_session(a.timeout, a.insecure)
    fs, start = [], time.time()
    try:
        resp = session.get(target, timeout=a.timeout, allow_redirects=True)
    except RequestException as e:
        sys.exit(f"{C.RED}[!] Cannot reach target: {e}{C.RST}")
    print(f"    HTTP {resp.status_code} — final URL: {resp.url}")
    base = f"{urlparse(resp.url).scheme}://{urlparse(resp.url).netloc}"
    check_https(session, target, fs)
    if urlparse(resp.url).scheme == "https" and host:
        check_tls(host, fs, a.timeout)
    check_headers(resp, fs); check_disclosure(resp, fs); check_cookies(resp, fs)
    check_methods(session, base, fs); check_sensitive_files(session, base, fs, a.delay)
    check_directory_listing(resp, fs)
    el = time.time() - start
    print_report(target, fs, el)
    if a.json:
        with open(a.json, "w", encoding="utf-8") as fh:
            json.dump({"tool": "WebScan-Lite v1.0", "target": target,
                       "final_url": resp.url, "scanned_at": datetime.now(timezone.utc).isoformat(),
                       "score": score(fs), "grade": grade(score(fs))[0], "findings": fs}, fh, indent=2)
        print(f"  JSON report written to: {a.json}\n")

if __name__ == "__main__":
    main()
```